Why is this code not creating a different count for the 2 different groups.

```
 let ci = 0;
        const incCount =
document.getElementById("incCount");
        const incCount_b =
document.getElementById("incCount_b");
        function inc() {
            ci = (ci >= 10000) ? 0 : ci + 1;
            update();
        }


function update() {
    incCount.textContent = ci;
    incCount_b.textContent = ci;
}

  const mouseover = function(event, d) {
    const subgroupName =
d3.select(this.parentNode).datum().key;
    const subgroupValue = d.data[subgroupName];
    inc()
    update()

    tooltip
        .html("Disciplinary action: " + subgroupName + "
<br>" + "Percentage: " + subgroupValue)
        .style("opacity", 1)
  }
```

Because both DOM elements are being updated from the same single variable:

JavaScript

and both of these lines use that same value:

```javascript
incCount.textContent = ci;
incCount_b.textContent = ci;
```

So they will **always display the same number**, by design.

What your code is currently doing

You have:

- One shared counter: `ci`
- One update function that writes `ci` into both elements

So both groups are *not independent counters* — they're just two views of the same state.

Why it feels like it "should" be different

You probably expect:

- Group A → its own count
- Group B → its own count

But right now there is **no separation of state per group**, so nothing can diverge.

How to fix it (separate counters per group)

You need **two variables**, or better: a structure keyed by group.

Option 1: two independent counters

```javascript
let ci_a = 0;
let ci_b = 0;
```

Then update separately depending on group.

Option 2 (better): object keyed by subgroup

```
const counts = {
  a: 0,
  b: 0
};
```

Then update based on subgroup:

```
function inc(group) {
    counts[group] = (counts[group] >= 10000) ? 0 : counts[group] + 1;
    update();
}

function update() {
    incCount.textContent = counts.a;
    incCount_b.textContent = counts.b;
}
```

Important bug in your current D3 handler

You also have this:

```
inc()
update()
```

But `inc()` already calls `update()`, so you're updating twice unnecessarily.

Cleaned-up version of your handler

```
const mouseover = function(event, d) {
    const subgroupName = d3.select(this.parentNode).datum().key;
    const subgroupValue = d.data[subgroupName];
```

```
        inc();

        tooltip
            .html("Disciplinary action: " + subgroupName + "<br>" + "Percentage: " +
            .style("opacity", 1);
    }
```

Core takeaway

Right now your “two groups” are not actually separate state machines — they are both reading from the same global variable.

If you want different counts, you must store **separate state per group**, not reuse one variable.

If you want, I can help you wire this directly into your D3 structure so each bar/segment maintains its own counter cleanly.

why is it not attaching to the specific rectangle even when I call it like this when drawing the rectangles:

```
.data(d => d)
  .join("rect")
    .attr("x", d =>  x(d.data.demographic))

    .attr("y", d => y(d[1]))
    .attr("height", d => y(d[0]) - y(d[1]))
    .attr("width",x.bandwidth())
    .attr("stroke", "grey")
  .on("mouseover", mouseover)
```

Because nothing in your current setup actually *binds a unique counter to each rectangle*. Even though D3 attaches the event handler to each `<rect>`, your handler is still reading

from shared external state.

Let's break down what's happening.

1. The key misunderstanding

This line:

JavaScript

does attach the handler to each rectangle, yes.

But inside your handler you are doing:

JavaScript

and `inc()` modifies:

JavaScript

That variable is:

- global
- shared across ALL rectangles
- not stored in the datum (`d`)
- not stored per element

So every rectangle click updates the same counter.

2. D3 is doing the binding correctly — your state is not

This part is fine:

JavaScript

and each rectangle *does* get its own `d`.

So inside `mouseover`, this is valid:

JavaScript

But you're ignoring it for state and using global `ci`.

3. Why it feels like it's "not attaching"

Because you're expecting something like:

> "this rectangle should have its own counter"

But your code never stores anything on:

- `d`
- the DOM element
- or a keyed object

So all rectangles are just calling:

JavaScript

4. Fix options (choose the model you want)

Option A — counter per rectangle (attach to datum)

Add a field to `d`:

```
.on("mouseover", function(event, d) {
    d.count = (d.count || 0) + 1;
```

```
    tooltip
        .html("Count: " + d.count)
        .style("opacity", 1);
})
```

Now each rectangle has its own memory.

Option B — counter per group (recommended for stacked/grouped charts)

If rectangles represent groups (like demographics), key it:

JavaScript

Then:

```
.on("mouseover", function(event, d) {
    const key = d.data.demographic;

    counts[key] = (counts[key] || 0) + 1;

    tooltip
        .html(`${key}: ${counts[key]}`)
        .style("opacity", 1);
})
```

Option C — attach directly to DOM element (D3 classic trick)

```
.on("mouseover", function(event, d) {
    this.__count = (this.__count || 0) + 1;

    tooltip
        .html(this.__count)
        .style("opacity", 1);
})
```

Now each rectangle stores its own state.

5. Small but important D3 issue in your snippet

This line:

JavaScript

only works if the parent selection is already structured correctly (nested data). If not, it can silently flatten or misbind data.

If rectangles seem “wrong,” the real issue may actually be upstream in how the stacked data is created.

6. Core takeaway

Attaching `.on("mouseover", mouseover)` is not the problem.

The problem is:

> You are not storing state per datum or per element — you're using one global variable (`ci`), so everything shares it.

If you want, paste how you're building the stacked data (`d3.stack` or similar). That’s often where the real “why aren’t rectangles behaving individually?” issue actually comes from.

Dont give me the answers. Help me understand why it is not working.

```
function inc(d) {
    let key = d.data.demographic ;
      ci[key] = (ci >= 10000) ? 0 : ci + 1;
      ci_b[key] = (ci_b >= 10000) ? 0 : ci_b + 1;
      console.log(d.data.demographic);
```

```
    update();
}
```

why is this function not extracting the data.demographic

The issue is not that D3 "can't extract `d.data.demographic`" — it's that your function is only *sometimes receiving a `d` that actually has a `.data` property.*

So the real question becomes: **what is `d` at the moment `inc(d)` runs?**

1. First thing to notice: `d` is not guaranteed what you think it is

You wrote:

JavaScript

That means you expect `d` to be passed in.

But earlier you likely call it like:

JavaScript

If that happens, then:

- `d === undefined`
- so `d.data` immediately breaks or becomes meaningless

So step one: **is `d` actually being passed?**

2. Even if `d` is passed — is it the right kind of `d`?

In stacked bar charts (which your code looks like), D3 typically gives you something like:

JavaScript

BUT that only exists if:

- you're inside a *stacked series structure*
- and you're on the *rect layer*, not the outer group

If you accidentally pass:

- the inner array `[y0, y1]`
- or a parent datum
- or a DOM event context

Then `d.data` will not exist.

3. The biggest hidden issue in your function

Look at this line:

JavaScript

This assumes:

d → has property `data` → which has property `demographic`

But in D3 stacks, `d` often looks like:

```
d = {
  0: y0,
  1: y1,
  data: {...}
}
```

OR sometimes just:

JavaScript

So if `d` is not the "stacks-with-data wrapper", then:

- `d.data` is undefined
- so `d.data.demographic` fails

4. Why your console.log might mislead you

You wrote:

JavaScript

If `d` is undefined or malformed:

- the function may crash before logging
- or log nothing useful depending on where failure occurs

5. The subtle conceptual issue

You are mixing two different D3 “levels”:

Level A (rect data)

This is what you think you have:

JavaScript

Level B (stack segment data)

This is what you *might actually have*:

JavaScript

If you're not exactly on Level A, your assumption breaks.

6. How to debug what is *actually happening*

Instead of assuming structure, inspect it:

```
function inc(d) {
    console.log("RAW d:", d);
    console.log("d.data:", d?.data);
}
```

Then ask yourself:

- Does `d` even have `.data`?
- Or is `demographic` somewhere else?

7. One more subtle trap (important)

If you are using:

JavaScript

then inside `mouseover`:

JavaScript

BUT if you later call:

JavaScript

you must ensure you're passing the SAME `d`, not recomputing or shadowing it.

Core insight

Nothing is wrong with "extracting `.data.demographic`" in JavaScript.

The real issue is:

At the moment your function runs, `d` is not consistently the object you think it is.

If you want, paste your `mouseover` function + where `inc(d)` is called, and I can help you trace exactly what shape `d` has at each step (that's usually where the confusion resolves fast).

why is d undefined

Because **you are calling `inc` without passing `d`**, so JavaScript has nothing to put into that parameter.

Let's connect the dots with your flow.

1. What you *expect*

You wrote:

JavaScript

So you expect:

"When I call `inc`, it receives the rectangle's data object `d`."

That only happens if you call it like:

JavaScript

2. What is actually happening

Earlier you likely have something like:

JavaScript

and inside:

```
function mouseover(event, d) {
    inc();   // ❌ no argument passed
}
```

So what happens is:

- `mouseover` receives `(event, d)` correctly
- but `inc()` is called with **no arguments**
- so inside `inc`, JavaScript sets:

JavaScript

3. Why JavaScript does this

JavaScript does NOT automatically "carry over" variables between functions.

This:

JavaScript

does not magically reuse the `d` from `mouseover`.

You must explicitly pass it.

4. Why this is especially confusing in D3

D3 *does* pass `d` into event handlers:

JavaScript

So you correctly get `d` there.

But once you call another function (`inc`), you are now in plain JavaScript rules again.

D3 is out of the picture.

5. The key mental model

Think of it like this:

- D3 gives `d` → only to the event handler
- Your handler → must forward `d` if another function needs it

Right now you're doing:

```
D3 → mouseover(d) ✓
mouseover → inc() ❌ (loses d)
```

6. Why you see "d is undefined"

Inside `inc` :

JavaScript

but you called:

JavaScript

So JavaScript sets:

JavaScript

That's it — no mystery beyond that.

7. The deeper takeaway (important for D3)

In D3, **data does not "flow" automatically between your functions**.

It only flows:

- from selection → event handler
- not between your own helper functions

So whenever you write helper functions like `inc`, ask:

> "Am I passing the data into this function, or assuming it exists?"

If you want, I can help you trace your full event chain (rect → mouseover → inc → update), because there's also a subtle state issue in your counters that will show up next.

when I do console.log(d.data.demographic) I get the demographic group that I am hovering over but why is the count not being specific for the groups even tho I get it on the console

Because you're mixing up "**reading the correct group**" with "**storing state per group.**" Those are two completely different things in your code.

You've already proven something important:

JavaScript

- ✔ This works → meaning `d` is correct
- ✔ You *are* identifying the correct group on hover

So the data side is fine.

The real problem: your counter is NOT tied to the group

Even though you *read* the group correctly, you are still updating a shared/global or incorrectly indexed value.

Look at the pattern you had earlier:

JavaScript

There are two conceptual issues here.

1. You are checking the WRONG variable in your condition

You wrote:

```
(ci >= 10000)
```

But `ci` is not the per-group value — it's your old global idea.

So every group is being evaluated using the same condition source instead of:

> "this specific group's count"

That alone guarantees shared behavior.

2. You are updating an object like an array

You switched to:

JavaScript

So now `ci` is an object (or intended to be).

But you're still treating it like a single number:

JavaScript

So what's actually happening is:

- you read the correct group (`key`)
- but you do NOT use `ci[key]` consistently in the math
- so values bleed across groups or become `NaN` / shared logic

3. The key conceptual mismatch

You currently have:

What you *think* you're doing:

> "each demographic group has its own counter"

What your code is actually doing:

> "I have one counter system, but I occasionally label parts of it with keys"

Those are not the same structure.

4. Why console.log misleads you

This is the subtle trap:

JavaScript

This proves:

✔ You are hovering the correct group

But it does NOT prove:

❌ You are storing the count per group correctly

So everything *feels* right visually, but state is still global.

5. The mental model that fixes this

You need to separate:

A. Identification (you already have this)

JavaScript

B. Storage (this is where the bug is)

Right now you accidentally still have a shared or inconsistent structure.

A correct mental model is:

> "Each key points to its own number, and only that number changes."

So the structure must behave like:

```
A → 1
B → 7
C → 3
```

Not:

```
one shared ci that sometimes uses A, B, or C
```

6. Why your current behavior looks "shared"

Even small mistakes like:

- using `ci` instead of `ci[key]`
- mixing number + object logic
- not initializing keys

will collapse everything into one shared result.

So every hover *appears* like it's updating one counter.

7. The key insight

You already solved the hardest part:

✔ You correctly identify the group with `d.data.demographic`

The missing piece is:

> You are not consistently using that group as the *index into your state*

If you want, I can walk you through a clean mental model of how D3 + object state should interact (it usually clears up 90% of these issues fast).